Thadomal Shahani Engineering College, Bandra

Department of Artificial Intelligence & Data Science

Name: Tanay Malvankar		Subject: ASD&D
T.E. - T1 Batch: T13 Sem: V	Roll No: 58	Date: 23/07/26

Experiment 1

LO2: Use git effectively for version control, source code management and collaborative management

Introduction to Git and GitHub:

1. Global Username Configuration

PowerShell

```
git config --global user.name "codexaras"
```

- **Purpose:** Sets the global Git username configuration. This ensures that every future code snapshot (commit) created on this machine is officially signed and tracked under the author profile codexaras.

2. Initial Email Configuration

PowerShell

```
git config --global user.email "your-email@example.com"
```

- **Purpose:** Sets an initial placeholder email address for the global configuration.

3. Username Reinforcement

PowerShell

```
git config --global user.name "codexaras"
```

- **Purpose:** Re-executed to verify and lock in the global author username

profile configuration.

4. Corrected Email Configuration

PowerShell

```
git config --global user.email "aravpalod2@gmail.com"
```

- **Purpose:** Overwrites the placeholder email configuration with the correct personal email address (tanaymalvankar01@gmail.com) ensuring lab submissions and cloud platforms correctly identify the committer.

5. Listing Active Configurations

PowerShell

```
git config --list
```

- **Purpose:** Displays the full active configuration profile of the local Git environment. It was used here to verify that user.name=codexaras and user.email = tanaymalvankar01@gmail.com were successfully saved in the system.

6. Local Repository Initialization

PowerShell

```
git init
```

- **Purpose:** Initialized a brand new, empty local Git repository in the active directory path (C:/Users/Lab 203/.git/). This creates the hidden tracker system files required to begin version control on the local machine.

7. Erroneous Repository Clone Attempt

PowerShell

```
git clone https://github.com/codexaras/experiment 1.git
```

- **Purpose:** Attempted to download a remote project structure. This command failed with a fatal: repository not found error due to a syntax typo where an underscore (experiment_1) was used instead of a hyphen.

8. Successful Remote Repository Cloning

PowerShell

```
git clone https://github.com/codexaras/experiment-1.git
```

- **Purpose:** Downloads the target online repository directly onto the local workstation machine. The console generated a warning stating You appear to have cloned an empty repository, confirming the operation was successful and a clean workflow directory named experiment-1 was successfully linked to GitHub.

9. Repository Status Verification & Directory Scope Check

PowerShell

```
git status
```

- **Purpose:** Checks the current state of the active repository timeline. Because this was inadvertently run inside the system's User Home folder (C:\Users\Lab 203) instead of inside the project subfolder, Git scanned the entire user directory.

- Analysis of Console Output:
    - Permission Warnings: The terminal generated multiple Permission denied warnings because Git attempted to scan protected Windows system directories (such as Application Data and Local Settings).
    - State Context: The output On branch master and No commits yet shows that the root User Home directory had an unconfigured Git tracking context initialization.
    - Untracked Files (Red Output): Git listed the entire contents of the computer's user folder—including application settings, system configurations, internal caches, and the experiment-1/ project folder—identifying them all as loose, untracked modifications. This step highlighted that the directory location needed to be updated using a change-directory command before continuing.

10. Empty Script File Initialization

PowerShell

```
New-Item main.py -ItemType File
```

- **Purpose:** Dynamically creates a brand-new, empty file named main.py directly inside the targeted experiment-1 project directory (C:\Users\Lab 203\experiment-1). The console output displays the file metadata, confirming a file length of 0 bytes.

11. Project Content Staging

PowerShell

```
git add .
```

- **Purpose:** Stages all newly created or modified assets within the project workspace folder. This marks the newly created empty main.py file as ready to be packaged into the subsequent repository history snapshot.

12. Root Commit Execution

PowerShell

```
git commit -m "Add initial files for experiment-1"
```

- **Purpose:** Saves the staged baseline changes permanently to the local Git timeline history ledger. The terminal output notes the creation of a master root-commit tagged with the unique identifier 8cc79bb.

13. Failed Remote Syncing (Authentication Conflict)

PowerShell

```
git push -u origin main
```

- **Purpose:** Attempted to upload the local commit timeline up to the remote GitHub cloud server. The execution failed with a fatal: 403 Forbidden error because the system attempted to authenticate using a cached credential set which lacks permission to write to the target repository.

14. Successful Remote Integration & Re-Authentication

PowerShell

```
git push -u origin main
```

- **Purpose:** Re-executed the cloud sync operation after removing the invalid cached system tokens. The console prompted browser-based user authorization, established the proper credentials, securely uploaded the data

packages to GitHub, and set up the local main branch to track the remote origin/main server track.

15. Redundant File Creation Attempt

PowerShell

```
New-Item main.py -ItemType File
```

- **Purpose:** An accidental attempt to re-create the base script file. The operating system blocked execution and threw a red WriteError: IOException warning declaring that the target file main.py already exists in that path.

16. Redundant Staging and Commit Checks

PowerShell

```
git add .

git commit -m "Add initial files for a second test"
```

- **Purpose:** Attempted to compile a secondary snapshot out of caution. Git evaluated the directory tracking tree, recognized no new content changes had occurred since the last operation, and returned nothing to commit, working tree clean.

17. Final Sync Verification

PowerShell

```
git push -u origin main
```

- **Purpose:** Checks the synchronization status between the local workstation and the GitHub cloud server. The output Everything up-to-date confirms that both storage contexts match completely.

18. Check Unstaged File Differences

PowerShell
```
git diff
```

- **Purpose:** Analyzes the directory for modified file text. Since no text modifications had been written yet, the command returned blank, confirming no local drift existed.

19. Local Branch Enumeration

PowerShell
```
git branch
```

- **Purpose:** Lists available development branches. The output * main verified that only the default development timeline existed and was actively selected.

20. Isolated Feature Branch Creation

PowerShell
```
git branch feature-optimization
```

- **Purpose:** Creates a secondary, parallel development branch named feature-optimization. This creates an isolated sandbox environment to safely script changes without altering the stable main timeline.

21. Branch Environment Selection

PowerShell
```
git switch feature-optimization
```

- **Purpose:** Switches the active working environment from main to the newly established feature-optimization timeline branch.

22. Redundant Duplicate Branch Creation Conflict

PowerShell
```
git checkout -b feature-optimization
```

- **Purpose:** An attempt to create and switch branches simultaneously. Git blocked the action and generated a fatal error message stating that a branch named feature-optimization already exists.

23. Preemptive Branch Merging Test

PowerShell
```
git merge feature-optimization
```

- **Purpose:** Attempts to pull developments from the feature branch into the active workspace tracking. The terminal returned Already up to date because no new commits had been written to either timeline.

24. Blocked Branch Deletion Attempt

PowerShell
```
git branch -d feature-optimization
```

- **Purpose:** Attempted to delete the feature branch. Git blocked the process with an error stating cannot delete branch... used by worktree, preventing the user from deleting a branch they are actively standing inside.

25. Return to Primary Timeline Context

PowerShell
```
git switch main
```

- **Purpose:** Switches the workspace target environment back to the primary main branch timeline so subsequent maintenance actions can be performed.

26. Successful Feature Branch Deletion Clean-up

PowerShell
```
git branch -d feature-optimization
```

- **Purpose:** Safely deletes the now-vacated feature-optimization branch. The output logs that the branch was cleanly dropped at baseline point 8cc79bb.

27. Redundant Remote Mapping Conflict

PowerShell
```
git remote add origin https://github.com/codexaras/experiment-1.git
```

- **Purpose:** Attempted to establish a new link to the online repository. The console threw an error: remote origin already exists because the initial cloning step had already automated this path connection mapping.

28. Remote Sync Verification

PowerShell
```
git push -u origin main
```

- **Purpose:** Uploads local timeline states up to GitHub to confirm consistency,

returning Everything up-to-date.

29. Remote Content Integration Sync

PowerShell
```
git pull
```

- **Purpose:** Checks the cloud server for new changes and integrates them. The response Already up to date verifies no remote changes had been submitted by external sources.

30. Remote Connection URL Inspection

PowerShell
```
git remote -v
```

- **Purpose:** Lists the explicit fetch and push network URLs mapped to the origin identifier, confirming secure tracking links lead straight to codexaras/experiment-1.git.

31. Simplified History Tracking Output

PowerShell
```
git log --oneline
```

- **Purpose:** Generates a compact list of history records. The console verified that local main, cloud origin/main, and head pointers all sat uniformly at root baseline commit 8cc79bb.

32. Staging Revocation Pathspec Failure

PowerShell
```
git restore --staged code.py
```

- **Purpose:** Attempted to pull a file out of the staging context area. The operation failed with a pathspec... did not match any file(s) error because no file named code.py existed anywhere within the repository folder.

33. Successful Script Staging Revocation

PowerShell
```
git restore --staged main.py
```

- **Purpose:** Correctly target-unstages main.py, removing it from the index queue without affecting its code content.

34. Uncommitted Change Discard Reversion

PowerShell
```
git restore main.py
```

- **Purpose:** Wipes out any unstaged text modifications inside main.py, forcing it to instantly match its baseline saved properties exactly as recorded in the history ledger.

35. Script Creation with Inline Values

PowerShell
```
New-Item main.py -ItemType File -Value 'print("Hello, World!")'
```

- **Purpose:** Creates a python script file (main.py) directly inside the experiment-1 project directory and populates it instantly with a simple hello world print instruction. The terminal metadata shows a length of 22 bytes, confirming the text data was successfully written to the disk.

36. Workspace Change Staging

PowerShell
```
git add .
```

- **Purpose:** Instructs Git to scan the local folder for modifications and stage the newly updated main.py script file, prepping it to be recorded in the local historical ledger.

37. Script Architecture Commit Execution

PowerShell
```
git commit -m "Add simple hello world script"
```

- **Purpose:** Packages the staged script changes and saves them permanently into a local history snapshot. The terminal logs that the commit was assigned the unique tracking hash ID 21904fd, recording 1 file modification and 1 line insertion.

38. Final Production Remote Syncing

PowerShell
```
git push origin main
```

- **Purpose:** Uploads the local script history timeline up to the remote GitHub cloud server repository. The terminal tracks the data packing progression and confirms a successful push

update, syncing the main timeline drift from previous position 6402268 to the newly established tracking peak 21904fd.

Conclusion:

This experiment successfully demonstrates the foundational workflow of Git version control by configuring user credentials, initializing local environments, and managing baseline code repositories. Through hands-on execution, core lifecycle operations—including file staging, history tracking logs, branching sandboxes, and timeline rollbacks—were systematically practiced and validated. Ultimately, the local project tracking structure was seamlessly connected and synchronized with the remote GitHub cloud interface, ensuring a reliable system for distributed engineering collaboration and project backup.